ATHANASIOS GOURDOMICHALIS
Numerical Analysis Python Programs
Newton-Raphson Method for Solving Non-Linear Equations

This document provides:

- **A short intro to the Newton-Raphson Method**
- **The program's problem statement (assignment prompt and requirements)**
- **The code description and implementation details.**

1. Introduction to Newton-Raphson Method

The **Newton-Raphson (N-R) method** is a premier **iterative technique** for **determining** the **real roots** of a **non-linear equation** $\boldsymbol{f(x) = 0}$. Originating **geometrically** from the **tangent line drawn** to the **curve** at $\boldsymbol{(x_n, f(x_n))}$ and **analytically** from the **first-order Taylor polynomial expansion**, the algorithm updates **successive approximations** via:

$$\boldsymbol{x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)} \equiv g(x_n)}$$

This scheme can be viewed as a **specialized application** of **Fixed-Point Iteration** $\boldsymbol{x = g(x)}$ where the **iteration function** is defined as $\boldsymbol{g(x) = x - \frac{f(x)}{f\prime(x)}}$.

- **Convergence Condition:**

If $\boldsymbol{f \in C^2[a, b]}$ and $\mathbf{x}^*$ is a **simple root** such that $\boldsymbol{f(x^*) = 0}$ and $\boldsymbol{f'(x^*) \neq 0}$, then there **exists** an **interval** $\boldsymbol{[x^* - \epsilon, x^* + \epsilon]}$ around **the root** such that for any **initial estimate** $\boldsymbol{x_0 \in [x^* - \epsilon, x^* + \epsilon]}$, the sequence $\{\boldsymbol{x_n}\}$ converges to $\boldsymbol{x^*}$.

- **Order of Convergence:**

Since $\boldsymbol{g'(x^*) = 1 - \frac{[f\prime(x^*)]^2 - f(x^*)f\prime\prime(x^*)}{[f\prime(x^*)]^2} = 0}$, the **method exhibits quadratic convergence** $\boldsymbol{(p = 2)}$ for **simple roots**:

$$\boldsymbol{\lim_{n\to\infty} \frac{|e_{n+1}|}{|e_n|^2} = \frac{|f''(x^*)|}{2|f'(x^*)|} = C}$$

If the root x^* has **multiplicity** $\mathbf{m} > \mathbf{1}$, the **convergence rate degenerates to linear** $(\boldsymbol{p} = \mathbf{1})$ with **asymptotic error constant** $\boldsymbol{C} = \mathbf{1} - \frac{\mathbf{1}}{\boldsymbol{m}}$.

2. Problem Statement

- **Non-linear Equation:**

$$f(x) = x^4 + 2x^3 - 7x^2 + 3 = 0$$

Its **first** analytical **derivative** is **given by**:

$$f'(x) = 4x^3 + 6x^2 - 14x$$

Out of the **four real roots** of the **polynomial**, exactly **two** are **positive**.

- **Requirements:**

1. **Root-Finding Function (`Newton-Raphson()`):** We implement the **Newton-Raphson method, taking** the **initial value** x_0, stopping **tolerance** `tol`, and **maximum iterations** `maxit` as **inputs**.
   The iteration terminates if $|x_{n+1} - x_n| <$ **tol, or** if the **number of iterations exceed maxit**.

2. **Automated Search for Both Positive Roots:** We **execute** the **function iteratively** for **initial points** $x_0 = i \cdot 0.1$ $(i = 0, 1, 2, \ldots)$ with $\texttt{tol} = 10^{-4}$ and $\texttt{maxit} = 15$.
   Solutions **matching within three decimal places** are **treated** as **identical**.
   Once **two distinct positive roots** are **identified**, the search **halts** and **displays both roots**.

3. **Convergence Analysis and Visualization for $x_0 = 1.4$:**
   We compute the **root,** starting from $x_0 = 1.4$ as an **initial approximation**, and **we plot the progression of consecutive errors** $e_n = |x_{n+1} - x_n|$ across the **iterations**.

3. Code Description & Implementation Details

- **Initial Setup & Declarations:**
  - We import `numpy` and `matplotlib.pyplot`.
  - Both the **function** $f(x) = x^4 + 2x^3 - 7x^2 + 3$ and **its derivative** $f'(x) = 4x^3 + 6x^2 - 14x$ are declared as **Python functions**.

- **The Iteration Function (`Newton-Raphson()`):**
  - At **each step** i, the **successive point** $x_{\text{next}} = x_0 - \frac{f(x_0)}{f'(x_0)}$ is evaluated.
  - The **step error** $|x_{\text{next}} - x_0|$ is **computed** and **recorded** into the **errors list**.
  - If $|x_{\text{next}} - x_0| <$ `tol` is true, the loop terminates early via break; otherwise, x_0 updates to x_{next}.
  - The function returns:
    the converged root and the list of step errors.

- **Systematic Search of Positive Roots:**
  - A while loop iterates over $x_0 = i \cdot 0.1$.
  - If $f'(x_0) == 0$ (e.g., at $x_0 = 0$), the point is bypassed to prevent division-by-zero errors (`ZeroDivisionError`).
  - The first root found is appended to `x_i`.
    The search continues until a second root differing by more than $\texttt{tol} = 10^{-4}$ is located.
  - Discovered Roots:

    $\texttt{Root 1} \approx 1.6180 \quad (1.61803399)$

    $\texttt{Root 2} \approx 0.7913 \quad (0.79128785)$

- **Convergence Study for $x_0 = 1.4$:**
  Executing from $x_0 = 1.4$ converges to the simple root $x^* \approx 1.6180$ in 5 iterations:
  - Iteration 1: $e_1 \approx 4.4337 \cdot 10^{-1}$
  - Iteration 2: $e_2 \approx 1.6744 \cdot 10^{-1}$
  - Iteration 3: $e_3 \approx 5.2566 \cdot 10^{-2}$
  - Iteration 4: $e_4 \approx 5.2714 \cdot 10^{-3}$
  - Iteration 5: $e_5 \approx 5.1418 \cdot 10^{-5}$

- Evaluating the ratios reveals: $e_5/e_4^2 \approx 1.8504 > 1$, which confirms quadratic convergence ($p = 2$) and closely matches the theoretical asymptotic constant:

$$C = \frac{|f''(x^*)|}{2|f'(x^*)|} = \frac{|12(1.6180)^2 + 12(1.6180) - 14|}{2|4(1.6180)^3 + 6(1.6180)^2 - 14(1.6180)|}$$

$$\approx \frac{36.8328}{20.0000} \approx 1.8416$$

- **Error Curve Visualization:**
  - Using `matplotlib.pyplot`, the consecutive errors are plotted against iteration indices with circular markers (`marker='o'`), integer tick marks on the horizontal axis (`plt.xticks`), a grid, and legends in both linear and logarithmic scales (`plt.yscale('log')`).